

Supplementary Material

A Multi-Scale Feature Fusion Network for Fine-Grained Soil Texture Classification

Boqun Li *et al.*

This supplementary document accompanies the main manuscript and provides the full protocol for the on-device latency benchmark reported in Section 4.5 and Table 7 of the paper. The protocol is made available so that the reported figures, including the MSFFN+KD latency of 61.33 ms on the CPU and 22.10 ms on the GPU along with the EfficientNet-Lite0 and EfficientNet-Lite2 reference baselines, can be reproduced by any reviewer or third party.

1 Mobile On-Device Inference Benchmark Protocol

This document describes the standardized protocol used to measure on-device single-frame inference latency for the soil-texture classification models reported in the paper. It is intended to make the benchmark numbers, such as the MSFFN+KD GPU mean latency of 22.10 ms and throughput of 45.3 FPS on the OnePlus 12, **reproducible** by any reviewer or third party.

1.1 Test device

Item	Specification
Device	OnePlus 12, PJD110, global ROM
SoC	Qualcomm Snapdragon 8 Gen 3
CPU	1×Cortex-X4 @ 3.3 GHz + 5×A720 + 2×A520
GPU	Adreno 750
OS	Android 14, ColorOS 14
RAM	16 GB LPDDR5X
Battery state	≥ 60 % under plugged-in charging conditions
Ambient temperature	22 ± 2 °C
Initial device temp	< 32 °C, verified via <code>dumpsys thermal</code> service before execution

1.2 Software stack

Item	Specification
MediaPipe Tasks Vision SDK	0.10.x, as specified in <code>app/build.gradle</code>
TensorFlow Lite	bundled by MediaPipe, with XNNPACK enabled by default for the CPU delegate
Delegates compared	CPU delegate via XNNPACK and GPU delegate via OpenGL ES
Inference mode	<code>RunningMode.LIVE_STREAM</code> , representing continuous camera-frame classification

Item	Specification
Models	<code>assets/efficientnet-lite0.tflite</code> , <code>assets/efficientnet-lite2.tflite</code> , <code>assets/soil_texture_model_with_metadata.tflite</code> , which undergoes post-training dynamic-range quantisation

1.3 Sample population

- All measurements use **the held-out test images / video clips collected from the same field campaigns as the training set**, never the training set itself.
- For the latency table reported in Section 4.5 of the paper, each model-delegate cell is exercised for ≥ 60 seconds of continuous live-camera inference after warm-up; the resulting per-cell sample count N ranges from 635 to 4263, depending on the achievable frame rate, as shown in Table 7 of the main manuscript.
- Frames are taken from the live camera preview rather than from pre-decoded files, to ensure the measurement reflects the end-to-end cost, which spans image acquisition, MPImage build, inference, and result parsing.

1.4 Warm-up

- The first `WARMUP_FRAMES=30` inferences after the helper is initialised are marked `IsWarmup=true` in the CSV log and **excluded** from latency statistics. Warm-up covers JIT compilation, GPU shader programme caching, and any first-time tensor allocation.
- After warm-up, we wait an additional 5 seconds idle before starting the timed window, to let the device’s DVFS settle.

1.5 Measurement procedure

1. Cold-launch the SoilIntelligent APK.
2. Open the camera tab, ensure the selected model is the one to be measured, such as `EfficientNet-Lite0`, `EfficientNet-Lite2`, or `Custom` representing our MSFFN+KD model, and select the desired delegate, namely the CPU or GPU.
3. Point the camera at a static soil patch under stable indoor illumination.
4. Hold steady for ≥ 60 seconds. During this period the helper streams inference requests as fast as the camera produces frames, which is approximately 30 FPS from the camera, though the latency measured is per-inference rather than per-frame.
5. Export the CSV log via `adb pull`:

```
adb shell run-as com.google.mediapipe.examples.imageclassification \
  cat files/inference_latency_logs.csv > inference_latency_logs.csv
```

6. Repeat steps 2–5 with the other delegate / model configuration.
7. Between configurations, let the device idle for ≥ 5 minutes so it returns to baseline temperature, which is verified by `dumpsys thermal` service, before the next session.

1.6 CSV Schema Version 2

`inference_latency_logs.csv` is written by `ImageClassifierHelper.kt` with the following header:

```
Timestamp,SessionId,SampleIdx,IsWarmup,ModelIndex,DelegateIndex,DelegateName,Mode,
InferenceTimeMs
```

Column	Meaning
Timestamp	<code>System.currentTimeMillis()</code> at log time, representing milliseconds since the epoch
SessionId	Random 8-char UUID per helper lifecycle, with one generated per app launch
SampleIdx	Monotonic 1-based sample counter within the session
IsWarmup	<code>true</code> if <code>SampleIdx ≤ WARMUP_FRAMES</code> , else <code>false</code>
ModelIndex	<code>Custom = 2</code> for our MSFFN+KD model, as defined in the helper companion constants
DelegateIndex	0 = CPU, 1 = GPU
DelegateName	Human-readable delegate string
Mode	One of <code>Image</code> , <code>Video</code> , <code>LiveStream</code>
InferenceTimeMs	<code>inferenceTimeMs</code> returned by MediaPipe Tasks, representing single-frame latency in milliseconds

1.7 Post-processing & statistics

We aggregate the CSV with the companion Python script `examples/customization/experiment_v2/aggregate_mobile_benchmark.py`, which:

1. Filters out all rows with `IsWarmup=true`.
2. Groups remaining rows by the tuple of `SessionId`, `ModelIndex`, `DelegateName`, and `Mode`.
3. Reports the **mean**, **median**, **p95**, **std**, and **N** of `InferenceTimeMs` per group, plus the corresponding `1000 / mean = FPS_mean` figure.
4. Optionally splits a session into 30-second windows to verify that no thermal throttling occurred mid-session, where a latency drift below 5 % across windows is considered stable.

1.8 Reported numbers in the paper

The numbers in Section 4.5 / Table 7 of the main manuscript correspond to the per-cell **mean** of all post-warm-up frames captured within a single ≥ 60 s session, taken under the protocol above. Six cells are reported: $\{\text{EfficientNet-Lite0, EfficientNet-Lite2, MSFFN+KD}\} \times \{\text{CPU, GPU}\}$. For example, the headline figure of 22.10 ms mean latency and 45.3 FPS for MSFFN+KD on the GPU aggregates $N = 1962$ post-warm-up samples drawn from a single continuous session, with $p_{95} = 29$ ms and standard deviation 4.10 ms.